Trabalho Prático: Otimização Evolutiva em Problemas Complexos

Universidade Federal do ABC
Computação Evolutiva e Conexionista
Relatório Técnico-Científico

Alunos:
Alexandre Keiti Fukamati
RA: 11202021788
a.fukamati@aluno.ufabc.edu.br

Gabriel de Oliveira Dalpian
RA: 11202230264
g.dalpian@aluno.ufabc.edu.br

28 de Abril de 2026

Resumo

Este trabalho apresenta a implementação e análise de um Algoritmo Genético aplicado ao Problema do Caixeiro Viajante, utilizando instâncias clássicas da TSPLIB. O problema consiste em encontrar a menor rota possível que visita todas as cidades exatamente uma vez e retorna à origem, sendo classificado como um problema de otimização combinatória de alta complexidade.

As soluções foram representadas como permutações de cidades, e a função de aptidão foi definida pela distância total da rota. O algoritmo utilizou seleção por torneio, elitismo, cruzamento do tipo *Order Crossover* e mutação por inversão. Foram realizadas múltiplas execuções independentes para cada instância, permitindo análise estatística dos resultados.

Os resultados indicam que o algoritmo apresenta bom desempenho em instâncias de menor dimensão, com baixos valores de *gap* em relação ao ótimo conhecido. Entretanto, observa-se aumento significativo do erro em instâncias maiores, evidenciando limitações na exploração do espaço de busca. As curvas de convergência mostram rápida melhoria inicial seguida de estabilização, sugerindo possível convergência prematura.

De forma geral, o trabalho demonstra a aplicabilidade de Algoritmos Genéticos em problemas combinatórios, ao mesmo tempo em que evidencia a importância do ajuste de parâmetros e da incorporação de estratégias adicionais para melhoria de desempenho.

Palavras-chave: Algoritmos genéticos, caixeiro viajante, TSP, otimização combinatória, computação evolutiva.

Sumário

1 Introdução **1**

2 Fundamentação Teórica **1**

2.1 Problema do Caixeiro Viajante 1

2.2 Algoritmos Genéticos 2

2.3 Representação das soluções 3

2.4 Função de *fitness* 3

2.5 Seleção por torneio 3

2.6 Cruzamento Order Crossover 4

2.7 Mutação por inversão 4

2.8 Elitismo 4

3 Metodologia **4**

3.1 Instâncias utilizadas 4

3.2 Leitura dos dados e matriz de distâncias 5

3.3 Configuração do Algoritmo Genético 5

3.4 Etapas do algoritmo 6

3.5 Avaliação dos resultados 7

4 Resultados e Discussão **7**

5 Conclusão **11**

6 Declaração de ética (IA Generativa) **11**

1 Introdução

Problemas de otimização combinatória estão presentes em diversas áreas da computação, engenharia, logística e planejamento. Em muitos desses problemas, o objetivo é encontrar a melhor solução possível dentro de um conjunto muito grande de combinações. À medida que o tamanho do problema cresce, a busca exaustiva por todas as possibilidades torna-se inviável, exigindo o uso de métodos aproximados e heurísticos.

Um dos problemas mais conhecidos dessa classe é o Problema do Caixeiro Viajante, ou *Travelling Salesman Problem* (TSP). Nesse problema, dado um conjunto de cidades e as distâncias entre cada par delas, deseja-se encontrar a menor rota que visite todas as cidades exatamente uma vez e retorne à cidade inicial. Apesar de sua formulação simples, o TSP é um problema computacionalmente difícil, pois o número de rotas possíveis cresce de forma fatorial com o número de cidades.

Nesse contexto, algoritmos evolutivos surgem como uma alternativa interessante para encontrar boas soluções em tempo computacional aceitável. Entre eles, os Algoritmos Genéticos se destacam por simular mecanismos inspirados na evolução natural, como seleção, reprodução, cruzamento, mutação e preservação dos indivíduos mais aptos. Essas técnicas permitem explorar diferentes regiões do espaço de busca e melhorar progressivamente a qualidade das soluções ao longo das gerações (Eiben & Smith, 2003; Goldberg, 1989).

Neste trabalho, propõe-se a implementação de um Algoritmo Genético para a resolução aproximada do Problema do Caixeiro Viajante. A implementação foi realizada em Python, utilizando as bibliotecas NumPy, Pandas e Matplotlib. Foram utilizadas instâncias clássicas da TSPLIB (Reinelt, 1991), com ótimos conhecidos, permitindo comparar as soluções encontradas pelo algoritmo com os melhores valores de referência.

O objetivo principal do trabalho é analisar o comportamento de um Algoritmo Genético aplicado a um problema combinatório complexo, observando sua convergência, qualidade das soluções, tempo de execução e escalabilidade em instâncias com diferentes números de cidades.

2 Fundamentação Teórica

2.1 Problema do Caixeiro Viajante

O Problema do Caixeiro Viajante consiste em determinar a menor rota fechada que passa por todas as cidades de um conjunto exatamente uma vez. Formalmente, dado um conjunto de n cidades e uma matriz de distâncias D, em que D_{ij} representa a distância

entre as cidades i e j, busca-se encontrar uma permutação das cidades que minimize o custo total do percurso.

A distância total de uma rota pode ser expressa por:

$$f(\pi) = \sum_{i=1}^{n-1} D_{\pi_i,\pi_{i+1}} + D_{\pi_n,\pi_1}$$

em que π representa uma permutação das cidades e o último termo corresponde ao retorno à cidade inicial.

O TSP é um problema clássico de otimização combinatória. Para n cidades, o número de rotas possíveis cresce aproximadamente de forma fatorial, tornando métodos exaustivos impraticáveis para instâncias maiores. Por isso, técnicas heurísticas e meta-heurísticas são frequentemente utilizadas para encontrar soluções aproximadas de boa qualidade.

2.2 Algoritmos Genéticos

Algoritmos Genéticos são métodos de busca e otimização inspirados no processo de evolução natural. Eles trabalham com uma população de soluções candidatas, chamadas indivíduos, que evoluem ao longo de gerações. Em cada geração, os indivíduos são avaliados por uma função de aptidão, e os melhores tendem a ter maior chance de gerar descendentes.

O ciclo básico de um Algoritmo Genético envolve:

- inicialização da população;
- avaliação dos indivíduos;
- seleção dos pais;
- aplicação de operadores de cruzamento;
- aplicação de mutação;
- formação da nova população;
- repetição do processo até uma condição de parada.

No caso do TSP, cada indivíduo representa uma rota possível. Como uma rota deve visitar cada cidade exatamente uma vez, a representação mais adequada é uma permutação dos índices das cidades.

2.3 Representação das soluções

Neste trabalho, cada solução é representada por uma permutação de inteiros. Cada inteiro corresponde a uma cidade, e a ordem dos elementos define a sequência de visitação. Por exemplo, uma solução para cinco cidades poderia ser representada como:

$$[3, 0, 4, 1, 2]$$

Essa permutação representa a rota:

$$3 \to 0 \to 4 \to 1 \to 2 \to 3$$

Essa representação garante que cada cidade seja visitada exatamente uma vez, desde que os operadores genéticos preservem a estrutura de permutação.

2.4 Função de *fitness*

A função de *fitness* utilizada corresponde à distância total da rota. Diferentemente de problemas de maximização, neste trabalho o objetivo é minimizar o valor da função de aptidão. Assim, quanto menor a distância total percorrida, melhor é o indivíduo.

A distância entre cidades foi calculada a partir das coordenadas bidimensionais das instâncias TSPLIB, utilizando a distância euclidiana arredondada:

$$d(i, j) = \text{nint}\left(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}\right)$$

em que (x_i, y_i) e (x_j, y_j) são as coordenadas das cidades i e j.

2.5 Seleção por torneio

A seleção por torneio consiste em sortear aleatoriamente um subconjunto de indivíduos da população e escolher, entre eles, aquele com melhor aptidão. No caso deste trabalho, o vencedor do torneio é o indivíduo com menor distância total.

Esse método permite controlar a pressão seletiva por meio do tamanho do torneio. Torneios maiores aumentam a chance de selecionar indivíduos muito bons, acelerando a convergência, mas também podem reduzir a diversidade da população. Torneios menores preservam mais diversidade, mas podem tornar o processo de evolução mais lento.

2.6 Cruzamento Order Crossover

Para o problema do TSP, o operador de cruzamento deve gerar filhos que continuem sendo permutações válidas. Por isso, foi utilizado o *Order Crossover* (OX), um operador adequado para problemas baseados em ordem.

O OX seleciona um segmento de um dos pais e o copia diretamente para o filho. Em seguida, as posições restantes são preenchidas com as cidades do segundo pai, respeitando a ordem em que aparecem e evitando repetições. Dessa forma, o operador preserva parcialmente a estrutura dos pais e mantém a validade da rota.

2.7 Mutação por inversão

A mutação por inversão consiste em selecionar dois pontos da rota e inverter a ordem das cidades entre eles. Esse operador é especialmente útil para o TSP, pois pequenas inversões podem remover cruzamentos entre arestas e reduzir a distância total da rota.

Por exemplo:

$$[1, 2, 3, 4, 5, 6] \rightarrow [1, 2, 5, 4, 3, 6]$$

A mutação contribui para manter a diversidade genética da população e reduzir o risco de convergência prematura para soluções locais.

2.8 Elitismo

O elitismo é uma estratégia que preserva os melhores indivíduos de uma geração, copiando-os diretamente para a próxima. Isso garante que a melhor solução encontrada até o momento não seja perdida devido aos operadores de cruzamento e mutação.

Neste trabalho, os melhores indivíduos de cada geração foram mantidos na população seguinte, aumentando a estabilidade do processo evolutivo e favorecendo a convergência.

3 Metodologia

3.1 Instâncias utilizadas

Foram utilizadas instâncias clássicas da TSPLIB, um conjunto de referência amplamente empregado para avaliação de algoritmos aplicados ao Problema do Caixeiro Viajante.

As instâncias selecionadas possuem diferentes números de cidades, permitindo avaliar o comportamento do algoritmo em problemas de tamanhos variados.

As instâncias utilizadas foram:

- `berlin52`, com 52 cidades e ótimo conhecido igual a 7542;
- `eil76`, com 76 cidades e ótimo conhecido igual a 538;
- `kroA100`, com 100 cidades e ótimo conhecido igual a 21282;
- `ch150`, com 150 cidades e ótimo conhecido igual a 6528;
- `a280`, com 280 cidades e ótimo conhecido igual a 2579.

Todas as instâncias consideradas utilizam coordenadas bidimensionais e distância euclidiana.

3.2 Leitura dos dados e matriz de distâncias

Os arquivos das instâncias foram lidos no formato TSPLIB. Para cada arquivo, foram extraídos o nome da instância, a dimensão do problema, o tipo de distância e as coordenadas das cidades.

A partir das coordenadas, foi calculada uma matriz de distâncias quadrada, em que cada posição D_{ij} representa a distância entre as cidades i e j. Essa matriz foi utilizada durante a avaliação das rotas, evitando o recálculo repetido das distâncias ao longo das gerações.

3.3 Configuração do Algoritmo Genético

A configuração experimental do Algoritmo Genético é apresentada na Tabela 1.

Tabela 1: *Parâmetros utilizados no Algoritmo Genético*

Parâmetro	**Valor**
Tamanho da população	120
Número de gerações	500
Taxa de cruzamento	0.8
Taxa de mutação	0.4
Tamanho do torneio	8
Tamanho da elite	8
Número de execuções por instância	25

Os parâmetros utilizados foram definidos de forma empírica, a partir da realização de diversos experimentos preliminares com diferentes configurações. Foram testadas variações no tamanho da população, taxas de cruzamento e mutação, bem como diferentes níveis de pressão seletiva e elitismo. A configuração apresentada corresponde àquela que proporcionou o melhor desempenho geral, considerando simultaneamente a qualidade das soluções encontradas, a estabilidade entre execuções e o tempo computacional.

Entretanto, essa configuração implica algumas características importantes no comportamento do algoritmo. O uso de um tamanho de torneio elevado ($k = 8$) introduz uma forte pressão seletiva, favorecendo a seleção recorrente dos melhores indivíduos e acelerando o processo de convergência. De forma semelhante, o elitismo com preservação de 8 indivíduos por geração garante que boas soluções não sejam perdidas ao longo do processo evolutivo. Contudo, a combinação desses dois fatores tende a reduzir a diversidade genética da população, podendo levar à convergência prematura para soluções subótimas, especialmente em instâncias de maior dimensão.

Por outro lado, a taxa de mutação relativamente alta (0.4) atua como um mecanismo de compensação, introduzindo variabilidade nas soluções e permitindo explorar novas regiões do espaço de busca. Essa escolha foi importante para evitar estagnação precoce do algoritmo, embora mutações excessivas possam, em alguns casos, dificultar o refinamento de soluções já promissoras.

Assim, a configuração adotada representa um compromisso entre exploração e exploração local do espaço de busca. Apesar de apresentar bom desempenho geral, os resultados indicam que o equilíbrio entre pressão seletiva, elitismo e mutação é um fator crítico, podendo impactar significativamente a qualidade das soluções e a capacidade de generalização do algoritmo em diferentes instâncias do problema.

3.4 Etapas do algoritmo

O algoritmo foi executado seguindo o ciclo evolutivo descrito abaixo:

1. geração de uma população inicial composta por rotas aleatórias;
2. cálculo da distância total de cada rota;
3. preservação dos melhores indivíduos por elitismo;
4. seleção de pais por torneio;
5. geração de filhos por *Order Crossover*;
6. aplicação de mutação por inversão;

7. substituição da população anterior pela nova população;
8. repetição do processo durante 500 gerações.

Ao final de cada geração, foram registrados o melhor valor de distância encontrado até o momento e a distância média da população. Esses valores foram utilizados para analisar a convergência do algoritmo.

3.5 Avaliação dos resultados

Para cada instância, o algoritmo foi executado 25 vezes independentemente. Essa estratégia foi adotada porque Algoritmos Genéticos são métodos estocásticos, ou seja, podem produzir resultados diferentes a cada execução.

As métricas avaliadas foram:

- melhor distância encontrada;
- pior distância encontrada;
- distância média;
- desvio padrão das distâncias;
- tempo médio de execução;
- consumo médio de memória;
- *gap* percentual em relação ao ótimo conhecido.

O *gap* percentual foi calculado por:

$$\text{Gap}(\%) = \frac{\text{distância encontrada} - \text{ótimo conhecido}}{\text{ótimo conhecido}} \times 100$$

Quanto menor o *gap*, mais próxima a solução encontrada está do ótimo conhecido.

4 Resultados e Discussão

Os resultados obtidos pelo Algoritmo Genético são apresentados na Tabela 2, bem como nas Figuras 1 e 2. A análise conjunta desses elementos permite avaliar tanto a qualidade das soluções encontradas quanto o comportamento do processo evolutivo ao longo das gerações.

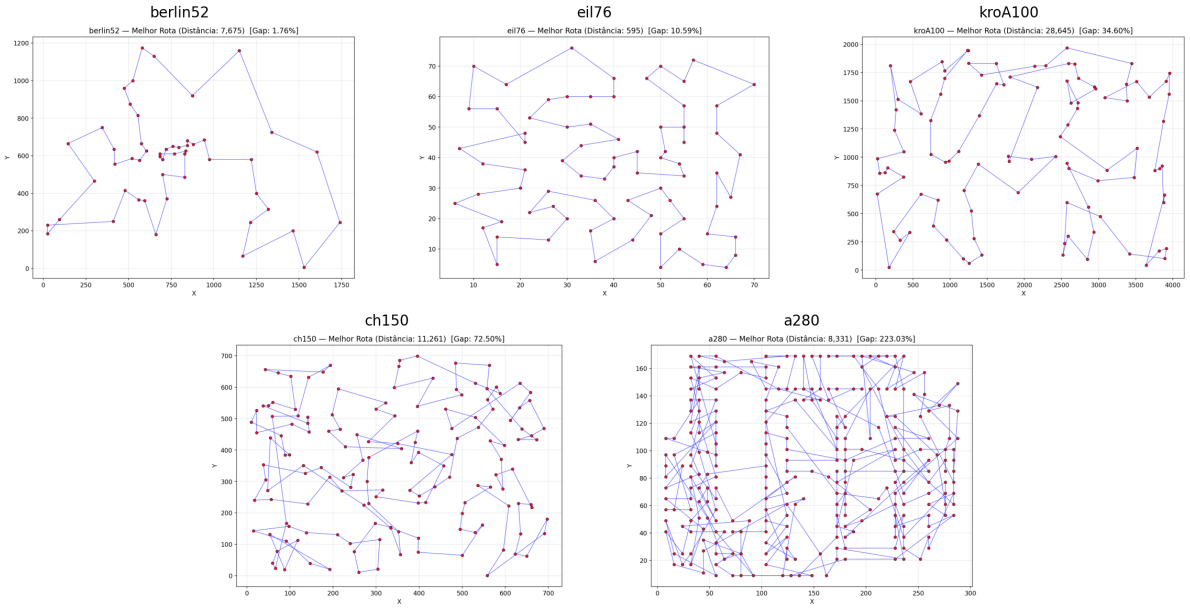

Figura 1: *Rota obtida para melhor execução em cada instância.*

A Figura 2 apresenta as curvas de convergência para todas as instâncias analisadas, considerando tanto o melhor indivíduo quanto a média da população. Observa-se que, em todas as instâncias, há uma queda acentuada da função de fitness nas primeiras gerações, indicando que o algoritmo rapidamente encontra soluções significativamente melhores do que as iniciais. Esse comportamento é característico de Algoritmos Genéticos, nos quais as primeiras iterações promovem grande exploração do espaço de busca.

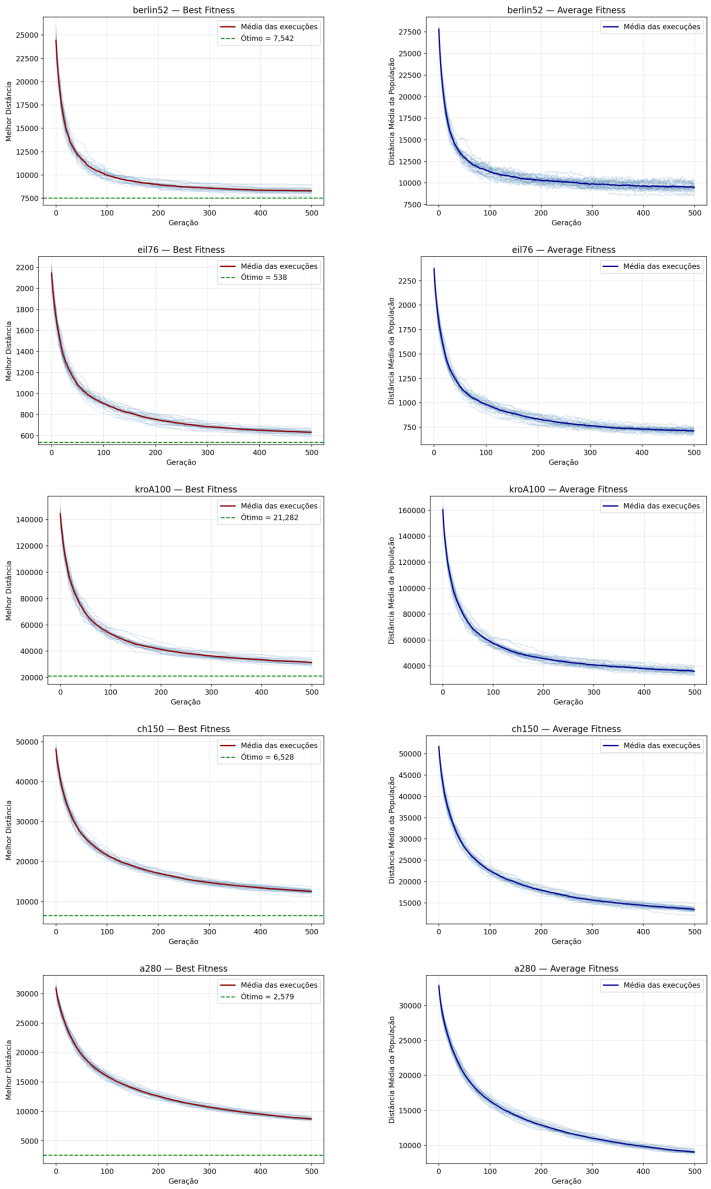

Figura 2: *Gráficos de convergência de melhor fitness e fitness médio em cada instância.*

À medida que o número de gerações aumenta, a taxa de melhoria diminui, e as curvas tendem a se estabilizar. Isso sugere que o algoritmo entra em uma fase de exploração local, refinando soluções já relativamente boas. Além disso, a proximidade entre as curvas de melhor fitness e fitness médio ao final das execuções indica uma redução da diversidade populacional, o que pode levar a convergência prematura em algumas instâncias.

A análise dos resultados da Tabela 2 mostra que o desempenho do algoritmo varia significativamente com o tamanho da instância.

Instância	Cidades	Ótimo	Melhor	Média	Desvio	Melhor Gap (%)	Gap Médio (%)	Tempo (s)	Memória (MB)
berlin52	52	7542	7675	8292.36	299.26	1.76	9.95	9.24	0.32
eil76	76	538	595	632.80	20.95	10.59	17.62	11.05	0.20
kroA100	100	21282	28645	31414.96	1584.00	34.60	47.61	12.70	0.25
ch150	150	6528	11261	12550.04	491.11	72.50	92.25	16.41	0.35
a280	280	2579	8331	8728.92	224.95	223.03	238.46	25.95	0.60

Tabela 2: *Resultados do Algoritmo Genético*

Para a instância `berlin52`, o algoritmo obteve um resultado bastante próximo do ótimo conhecido, com *gap* mínimo de apenas 1.76% e *gap* médio de 9.95%. Isso indica que, para problemas de menor dimensão, o Algoritmo Genético é capaz de encontrar soluções de alta qualidade de forma consistente. Para a instância `eil76`, o desempenho ainda é satisfatório, embora com maior afastamento do ótimo, apresentando *gap* médio de 17.62%. Já nas instâncias maiores, como `kroA100`, `ch150` e `a280`, observa-se um aumento expressivo do *gap*, chegando a valores superiores a 200% na instância `a280`. Esse comportamento evidencia a dificuldade crescente do problema à medida que o número de cidades aumenta, devido à explosão combinatória do espaço de busca.

A Figura 1 apresenta as melhores rotas encontradas para cada instância. Visualmente, observa-se que, nas instâncias menores, as rotas apresentam uma organização mais coerente, com menor ocorrência de cruzamentos entre arestas. Por outro lado, nas instâncias maiores, como `ch150` e `a280`, as rotas apresentam maior desorganização e presença de cruzamentos, o que contribui para o aumento da distância total.

Outro ponto relevante é o comportamento do tempo de execução e consumo de memória. Conforme mostrado na tabela, o tempo médio de execução cresce com o número de cidades, variando de aproximadamente 9 segundos para `berlin52` até cerca de 26 segundos para `a280`. Esse crescimento está diretamente relacionado ao custo de avaliação da população, que depende do tamanho da rota e do número de indivíduos. Já o consumo de memória apresentou crescimento moderado, mantendo-se abaixo de 1 MB em todas as instâncias, o que indica boa eficiência da implementação nesse aspecto.

De forma geral, os resultados demonstram que o Algoritmo Genético implementado é eficaz para encontrar boas soluções em instâncias de menor porte, apresentando convergência estável e baixo *gap*. Entretanto, para instâncias maiores, o algoritmo encontra maior dificuldade em se aproximar do ótimo global, o que sugere limitações relacionadas à exploração do espaço de busca e à manutenção da diversidade populacional.

Esses resultados indicam que, embora o modelo seja adequado como abordagem geral, melhorias podem ser exploradas, como ajuste mais fino dos hiperparâmetros ou aumento do número de gerações, que podem reduzir significativamente o *gap* em problemas de maior escala.

5 Conclusão

O presente trabalho teve como objetivo implementar e analisar um Algoritmo Genético aplicado ao Problema do Caixeiro Viajante, utilizando instâncias clássicas da TSPLIB como base experimental. A proposta permitiu explorar, de forma prática, os principais conceitos da computação evolutiva, incluindo representação de soluções, operadores genéticos e estratégias de seleção.

Os resultados obtidos demonstraram que o algoritmo é eficaz na obtenção de soluções de boa qualidade em instâncias de menor dimensão, apresentando baixos valores de *gap* e convergência estável ao longo das gerações. Entretanto, à medida que o número de cidades aumenta, observa-se uma degradação significativa no desempenho, evidenciada pelo crescimento do *gap* e pela dificuldade em aproximar-se do ótimo global.

A análise das curvas de convergência indicou que o algoritmo apresenta rápida melhoria nas primeiras gerações, seguida por estabilização, sugerindo possível convergência prematura. Esse comportamento pode estar relacionado à configuração dos hiperparâmetros, que impõem alta pressão seletiva e podem reduzir a diversidade da população.

De forma geral, o trabalho evidencia que, embora Algoritmos Genéticos sejam ferramentas robustas para problemas combinatórios, sua eficácia depende fortemente da escolha dos operadores e parâmetros. Como trabalhos futuros, sugere-se a incorporação de técnicas de busca local, bem como o ajuste mais refinado dos hiperparâmetros, visando melhorar a qualidade das soluções em instâncias de maior escala.

6 Declaração de ética (IA Generativa)

Ferramentas de inteligência artificial generativa foram utilizadas apenas como apoio para esclarecimento conceitual e organização textual do relatório. Todo o desenvolvimento do código, experimentos e análise dos resultados foi realizado pelos autores.

Referências Bibliográficas

Eiben, Agoston E., & Smith, James E. 2003. *Introduction to evolutionary computing.* Natural Computing Series. Springer.

Goldberg, David E. 1989. *Genetic algorithms in search, optimization, and machine learning.* Reading, MA: Addison-Wesley.

Reinelt, Gerhard. 1991. Tsplib - a traveling salesman problem library. *Informs j. comput.*, **3**, 376–384.